



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 22-b: Solución Mario Maker (Builder)

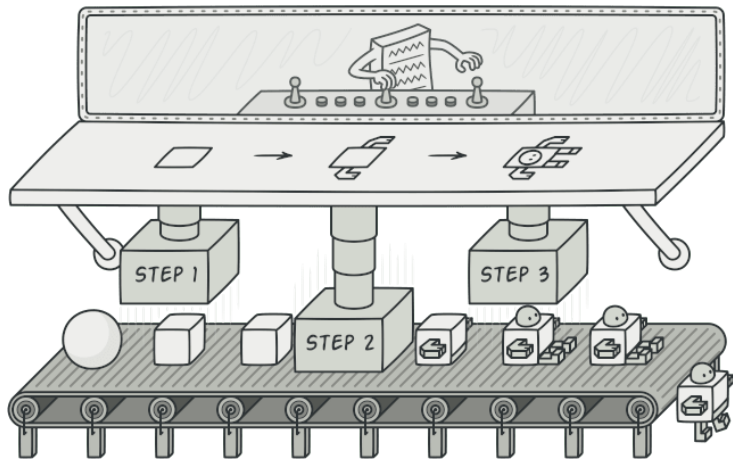
Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

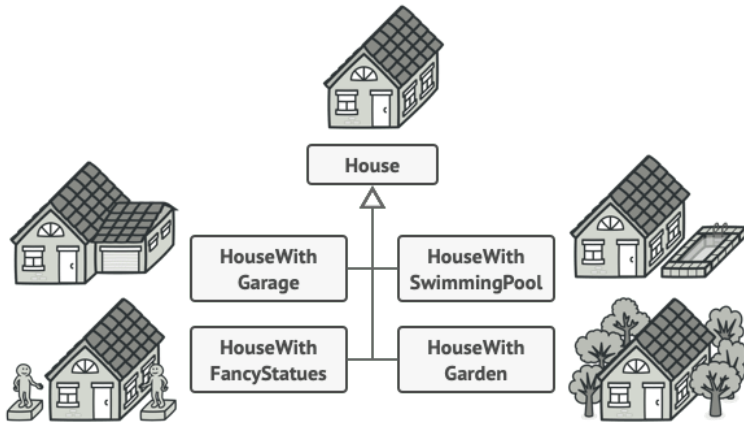
23 de Octubre, 2025

Mario Maker (Builder)

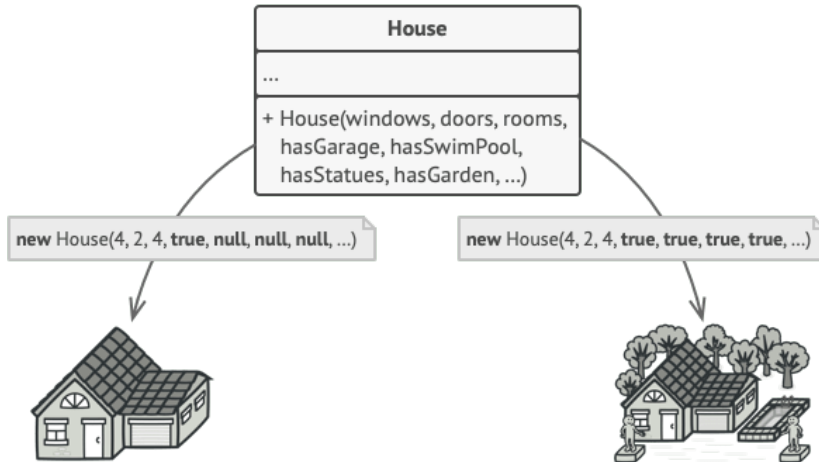
Mario Maker (Builder)



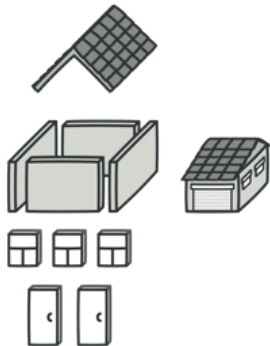
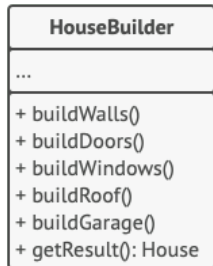
Mario Maker (Builder)



Mario Maker (Builder)



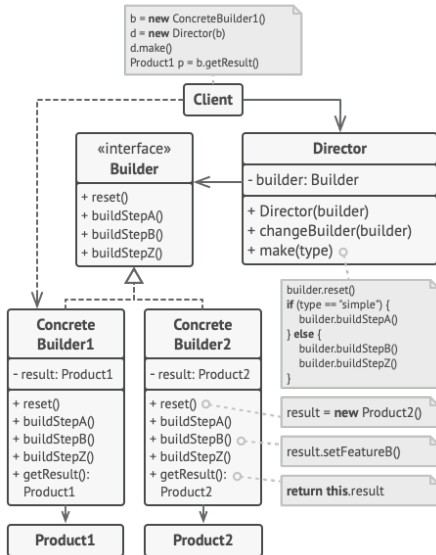
Mario Maker (Builder)



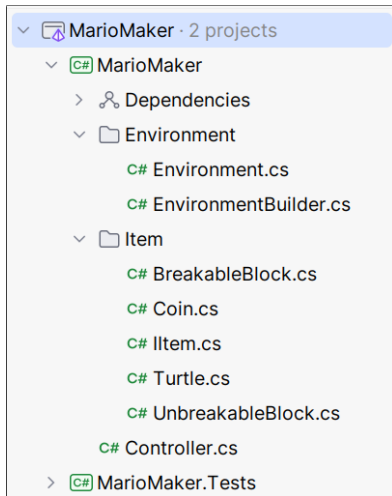
Mario Maker (Builder)



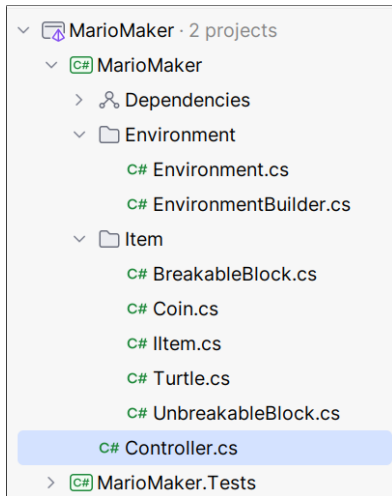
Mario Maker (Builder)



Mario Maker (Builder)



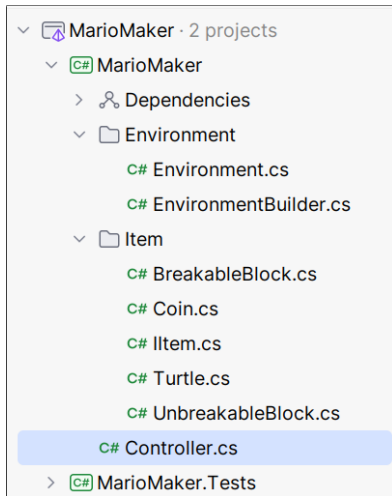
Mario Maker (Builder)



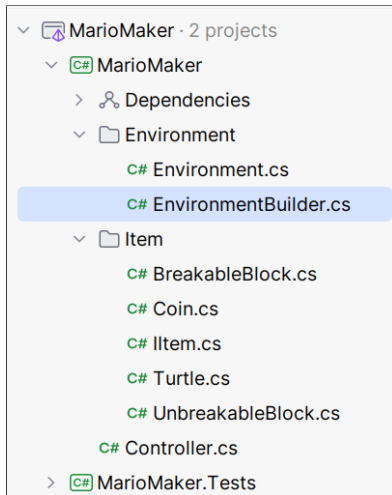
Mario Maker (Builder)

```
1 public static class Controller {  
2  
3     public static Environment BuildEnvironment(EnvironmentBuilder builder,  
4         string[] lines) {  
5         throw new NotImplementedException();  
6     }  
7  
8     public static EnvironmentBuilder BuildCloudEnvironmentBuilder() {  
9         throw new NotImplementedException();  
10    }  
11  
12    public static EnvironmentBuilder BuildLavaEnvironmentBuilder() {  
13        throw new NotImplementedException();  
14    }  
15  
16    public static string[] Export(Environment environment) {  
17        throw new NotImplementedException();  
18    }  
19 }
```

Mario Maker (Builder)



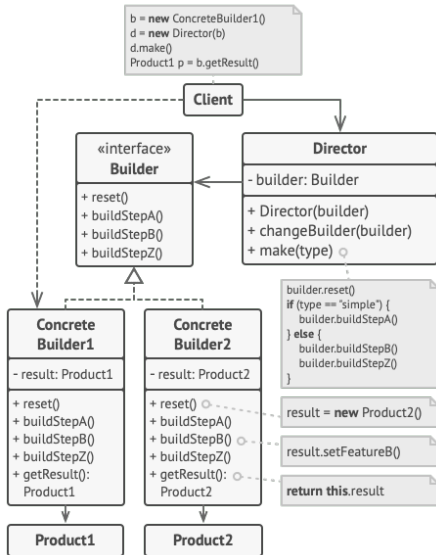
Mario Maker (Builder)



Mario Maker (Builder)

```
1 public class EnvironmentBuilder {
2     private Environment _environment;
3
4     public void Reset()
5         => _environment = new Environment();
6
7     public void SetDimensions(int height, int width)
8         => _environment.SetDimensions(height, width);
9
10    public void SetUnbreakableBlock(int row, int column)
11        => _environment.SetItem(row, column, new UnbreakableBlock());
12    public void SetBreakableBlock(int row, int column)
13        => _environment.SetItem(row, column, new BreakableBlock());
14    public void SetTurtle(int row, int column)
15        => _environment.SetItem(row, column, new Turtle());
16    public void SetCoin(int row, int column)
17        => _environment.SetItem(row, column, new Coin());
18
19    public Environment GetResult()
20        => _environment;
21 }
```

Mario Maker (Builder)



Mario Maker (Builder)

Parte a) Actualmente hay un programa para generar *templates* de niveles, los cuales son archivos de texto que representan una grilla. El número de líneas dentro del archivo representa la altura de la grilla y el largo de las líneas el ancho (puedes asumir que todas las líneas del archivo tienen el mismo largo).

Cada caracter (en caso de haber alguno) representa una entidad del ambiente:

- U: UnbreakableBlock.
- B: BreakableBlock.
- C: Coin.
- T: Turtle.

Implementa el método `BuildEnvironment(EnvironmentBuilder builder, string[] lines)` dentro de `Controller.cs`. Este método debe retornar un objeto del tipo `Environment` que sea construido a partir de un *template*, el cual está dado por el argumento `lines`.

Mario Maker (Builder)

```
1 public static class Controller {  
2  
3     public static Environment BuildEnvironment(EnvironmentBuilder builder,  
4         string[] lines) {  
5         throw new NotImplementedException();  
6     }  
7  
8     public static EnvironmentBuilder BuildCloudEnvironmentBuilder() {  
9         throw new NotImplementedException();  
10    }  
11  
12    public static EnvironmentBuilder BuildLavaEnvironmentBuilder() {  
13        throw new NotImplementedException();  
14    }  
15  
16    public static string[] Export(Environment environment) {  
17        throw new NotImplementedException();  
18    }  
19 }
```

Mario Maker (Builder)

```
1 public static Environment BuildEnvironment(EnvironmentBuilder builder, string
   [] lines) {
2     builder.Reset();
3
4     int height = lines.Length;
5     int width = lines[0].Length;
6     builder.SetDimensions(height, width);
7     for (int i = 0; i < height; i++)
8         for (int j = 0; j < width; j++)
9         {
10             if (lines[i][j] == 'U')
11                 builder.SetUnbreakableBlock(i, j);
12             if (lines[i][j] == 'T')
13                 builder.SetTurtle(i, j);
14             if (lines[i][j] == 'C')
15                 builder.SetCoin(i, j);
16             if (lines[i][j] == 'B')
17                 builder.SetBreakableBlock(i, j);
18         }
19
20     return builder.GetResult();
21 }
```

Parte b) Extiende el patrón Builder para generar niveles ambientados en el mundo de lava y en el mundo de nubes. Para ello implementa:

- `BuildLavaEnvironmentBuilder()`
- `BuildCloudEnvironmentBuilder()`

Por ahora, la única diferencia entre estos mundos es el tipo de tortugas que se encuentran en ellos. En el mundo de lava solo existen tortugas de lava y en el mundo de nube solo existen tortugas con alas.

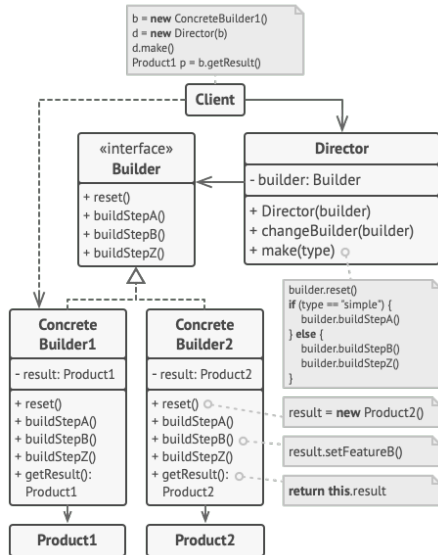
En específico, el método `Export()` de las tortugas del mundo de lava debe retornar “`LavaTurtle`” y las del mundo de nube debe retornar “`WingedTurtle`”.

Mario Maker (Builder)

Nos piden retornar builders para estos nuevos tipos de ambientes:

```
1 public static class Controller {
2
3     public static Environment BuildEnvironment(EnvironmentBuilder builder,
4         string[] lines) {
5         /* ... idem anterior ... */
6     }
7
8     public static EnvironmentBuilder BuildCloudEnvironmentBuilder(){
9         throw new NotImplementedException();
10    }
11
12    public static EnvironmentBuilder BuildLavaEnvironmentBuilder(){
13        throw new NotImplementedException();
14    }
15
16    public static string[] Export(Environment environment) {
17        throw new NotImplementedException();
18    }
19 }
```

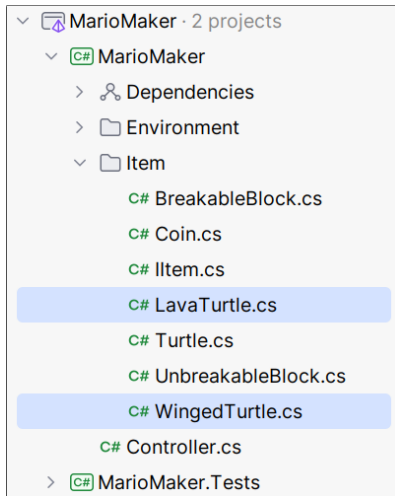
Mario Maker (Builder)



Mario Maker (Builder)

```
1 public class EnvironmentBuilder {
2     private Environment _environment;
3
4     public void Reset()
5         => _environment = new Environment();
6
7     public void SetDimensions(int height, int width)
8         => _environment.SetDimensions(height, width);
9
10    public void SetUnbreakableBlock(int row, int column)
11        => _environment.SetItem(row, column, new UnbreakableBlock());
12    public void SetBreakableBlock(int row, int column)
13        => _environment.SetItem(row, column, new BreakableBlock());
14    public void SetTurtle(int row, int column)
15        => _environment.SetItem(row, column, new Turtle());
16    public void SetCoin(int row, int column)
17        => _environment.SetItem(row, column, new Coin());
18
19    public Environment GetResult()
20        => _environment;
21 }
```

Mario Maker (Builder)



Mario Maker (Builder)

Esta es la tortuga de lava:

```
6 public class LavaTurtle : Turtle {  
7     public override string Export()  
8         => "LavaTurtle";  
9 }
```

Esta es la tortuga con alas:

```
11 public class WingedTurtle : Turtle{  
12     public override string Export()  
13         => "WingedTurtle";  
14 }
```

Mario Maker (Builder)

Esta es la tortuga de lava:

```
6 public class LavaTurtle : Turtle {  
7     public override string Export()  
8         => "LavaTurtle";  
9 }
```

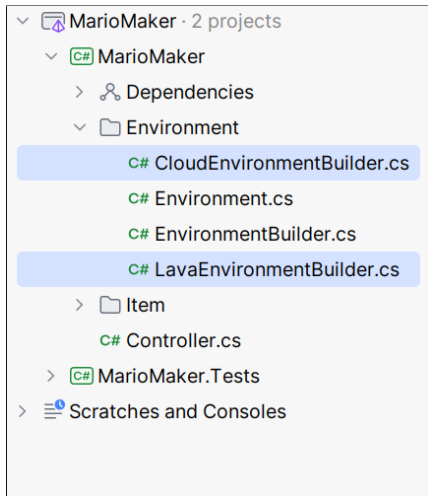
Esta es la tortuga con alas:

```
11 public class WingedTurtle : Turtle{  
12     public override string Export()  
13         => "WingedTurtle";  
14 }
```

... para que esto funcione tenemos que hacer que Export() virtual en Turtle:

```
1 public class Turtle : IItem {  
2     public virtual string Export() // Ahora es virtual!  
3         => "Turtle";  
4 }
```

Mario Maker (Builder)



Mario Maker (Builder)

Builder de lava (todo igual menos la tortuga):

```
1 public class LavaEnvironmentBuilder : EnvironmentBuilder {  
2     public override void SetTurtle(int row, int column)  
3         => Environment.SetItem(row, column, new LavaTurtle());  
4 }
```

Builder de nube (todo igual menos la tortuga):

```
6 public class CloudEnvironmentBuilder: EnvironmentBuilder {  
7     public override void SetTurtle(int row, int column)  
8         => Environment.SetItem(row, column, new WingedTurtle());  
9 }
```

Mario Maker (Builder)

Builder de lava (todo igual menos la tortuga):

```
1 public class LavaEnvironmentBuilder : EnvironmentBuilder {  
2     public override void SetTurtle(int row, int column)  
3         => Environment.SetItem(row, column, new LavaTurtle());  
4 }
```

Builder de nube (todo igual menos la tortuga):

```
6 public class CloudEnvironmentBuilder: EnvironmentBuilder {  
7     public override void SetTurtle(int row, int column)  
8         => Environment.SetItem(row, column, new WingedTurtle());  
9 }
```

... para que esto funcione tenemos que hacer cambios menores al builder base:

```
11 public class EnvironmentBuilder{  
12     protected Environment Environment; // Pasa a protected  
13     public virtual void SetTurtle(int row, int column) // Pasa a virtual  
14         => Environment.SetItem(row, column, new Turtle());  
15     /* ... el resto queda igual... */  
16 }
```

Mario Maker (Builder)

Ahora simplemente basta con retornar el builder apropiado en el Controller:

```
1 public static class Controller {  
2  
3     public static Environment BuildEnvironment(EnvironmentBuilder builder,  
4         string[] lines) {  
5         /* ... idem anterior ... */  
6     }  
7  
8     public static EnvironmentBuilder BuildCloudEnvironmentBuilder()  
9         => new CloudEnvironmentBuilder();  
10  
11    public static EnvironmentBuilder BuildLavaEnvironmentBuilder()  
12        => new LavaEnvironmentBuilder();  
13  
14    public static string[] Export(Environment environment) {  
15        throw new NotImplementedException();  
16    }  
17 }
```